Functions

PAGE NO.....
DATE / /

1) Parameters and Arguments

- Parameters : Variables listed inside the parenthesis in the function definition.
- Arguments : Values passed to the function when it is called.

E.g

```
def multiply(x, y)
    return x*y
result = multiply(4,5).
print(result).
```

2) Return statement.

-> The return statement is used to send a result back to the caller.

-> A function can return any type of data (e.g. number, strings, lists etc).

-> If there is no return statement, the function return None

E.g

```
def square(n)
    return n*n.
result = square(4)
print(result).
# output: 16
```

PAGE NO.....
DATE / /

3) Default Arguments.

```
def greet (name= "Guest");
    return f"Hallo, {name}"

print (greet ())          # output : Hallo, Guest.
print (greet ("Alice"))   # output : Hallo, Alice.
```

4) Keyword Argument.

```
def describe_pet (name, animal = "dog");
    return f"I have a {animal} named {name}."

print (describe_pet (name = "Max"))
    output : I have a dog named Max.

print (describe_pet (animal = "cat", name = "Whiskers"))
    output: I have a cat named Whisker.
```

5) Positional argument

*args (for positional argument).

**kwargs (for keyword arguments).

```
def sum_all (*args);
    return sum (args).
print (sum_all (1,2,3))  # output : 6.
```

```
def describe(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

describe(name="Alice", age=25).

# output name: Alice
  age: 25
```

6) Lambda functions

-> Lambda functions are small / anonymous functions defined using the Lambda keyword.

-> They are useful for short, one-time operations.

```
square = lambda x: x * x
print(square(5)) # output: 25.
```

7) Scopes of variable

```
x = 10
def update_x():
    global x
    x = 20.

update_x()
print(x) # output: 20.
```

-> local scope: Variables defined inside a function are local to that function.

-> Global Scope: Variables defined outside all functions are global and can be accessed everywhere

-> Use the global keyword to modify a global variable inside a functions.

PAGE NO.....
DATE / /

8) Recursion

-> A function can call itself. This is called recursion.

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)
print(factorial(5)) # output: 120.
```

9) Doc-strings

-> Docstrings are used to document functions. They are written as a string immediately after the function definition.

-> use help(function-name) to view the docstring

```
def add(a,b):
    """
    This function adds two numbers and returns the result.
    """
    return a+b.

print(help(add))
# output:
# This function adds two numbers and returns the results
```

Lambda Functions

PAGE NO.....
DATE / /

1) Lambda with Multiple Argument.

```
multiply = lambda x, y : x * y
print (multiply (3,4)) # output : 12.
```

2) Lambda with No argument.

```
greet = lambda : "Hallo, world!"
print (greet())   # output : Hallo, World!
```

3) Lambda functions with Built-in functions.

1) map() : applies a function to all items in an iterable

```
numbers = [1, 2, 3, 4]
doubled = map (lambda x : x * 2, numbers).
print (list (doubled)) # output : [2, 4, 6, 8]
```

2) filter()

: filters items in an iterable based on a condition.

```
numbers = [1, 2, 3, 4, 5, 6]
evens = filter (lambda x : x % 2 == 0, numbers).
print (list (evens))
```

3) sorted():

: sorts an iterable : You can use a lambda function as the key for custom sorting.

```
pairs = [(1,3), (4,1), (2,2)]
sorted_pairs = sorted (pairs, key = lambda
                                    x : x[1])
print (sorted_pairs).
```